
SALE : Low-bit Estimation for Efficient Sparse Attention in Long-context LLM Prefilling

Xiaodong Ji¹ Hailin Zhang¹ Fangcheng Fu² Bin Cui^{1,3}

Abstract

Many advanced Large Language Model (LLM) applications require long-context processing, but the self-attention module becomes a bottleneck during the prefilling stage of inference due to its quadratic time complexity with respect to sequence length. Existing sparse attention methods accelerate attention computation by skipping less significant regions of the attention map. However, these approaches typically perform coarse-grained inspection of the attention map, resulting in their suboptimal performance. In this paper, we propose **SALE**, a fine-grained sparse attention method that accelerates the long-context prefilling stage of LLM with negligible loss in model accuracy. SALE achieves fast and accurate fine-grained attention map estimation using low-bit quantized query-key products to approximate attention weights, followed by the application of a novel *Relative Attention Score* metric to assess the importance of query-key pairs. This design enables us to accurately identify important regions in the attention map, thereby constructing a highly sparse attention mask.

We implement a custom CUDA kernel in SALE optimized for hardware efficiency, reducing overhead to approximately 11% of the full attention latency. Notably, SALE requires no parameter training and can be seamlessly integrated into existing systems with trivial code modifications. Experiments on long-context benchmarks demonstrate that our method outperforms existing approaches in accuracy-efficiency trade-offs, achieving at least $3.36\times$ speedups on Llama-3.1-8B for se-

quences longer than 64K while maintaining model quality. Our code is available at <https://github.com/BirdChristopher/SALE>

1. Introduction

With the growing demand for ultra-long context understanding in complex applications such as long book summarization (Kryściński et al., 2022; Porwal et al., 2023; Chang et al., 2024), long document question-answering (Caciularu et al., 2023; Pang et al., 2022; Fan et al., 2019), and repository-level code completion (Wang et al., 2024a;b), state-of-the-art Large Language Models (LLM) are now equipped with increasingly longer context window (Grattafiori et al., 2024; Yang et al., 2025b; Team et al., 2025; DeepSeek-AI et al., 2025; Xu et al., 2025). Most LLMs employ a decoder-only Transformer architecture (Vaswani et al., 2017), where the self-attention module serves as the core component to enable powerful language understanding capabilities. However, during the prefilling stage of LLM inference, the self-attention module exhibits quadratic time complexity with respect to the number of input tokens. This makes it the primary performance bottleneck, as computational costs increase rapidly with longer contexts (Fu, 2024; Jiang et al., 2024; Lin et al., 2025).

In recent years, numerous research studies have attempted to accelerate prefilling by computing only the important regions of attention maps, based on the observation that attention maps in LLMs are significantly sparse (Deng et al., 2024). These methods, referred to as *sparse attention*, use *sparse masks* to indicate the specific regions of the attention map to be computed. Some sparse attention methods utilize sparse masks with static patterns, such as stride pattern (Child et al., 2019), window pattern (Zaheer et al., 2020; Beltagy et al., 2020), or streaming pattern (Xiao et al., 2024b; Han et al., 2024). However, static sparse masks often result in severe performance degradation, as the real sparse patterns of LLM attention maps are highly dynamic across various input contents (Lai et al., 2025; Jiang et al., 2024; Zhang et al., 2025e).

To adapt to such dynamism, several methods attempt to pre-

¹School of Computer Science & Beijing Key Laboratory of Software and Hardware Cooperative Artificial Intelligence Systems, Peking University ²School of Artificial Intelligence, Shanghai Jiao Tong University ³Institute of Computational Social Science, Peking University(Qingdao). Correspondence to: Bin Cui <bin.cui@pku.edu.cn>.

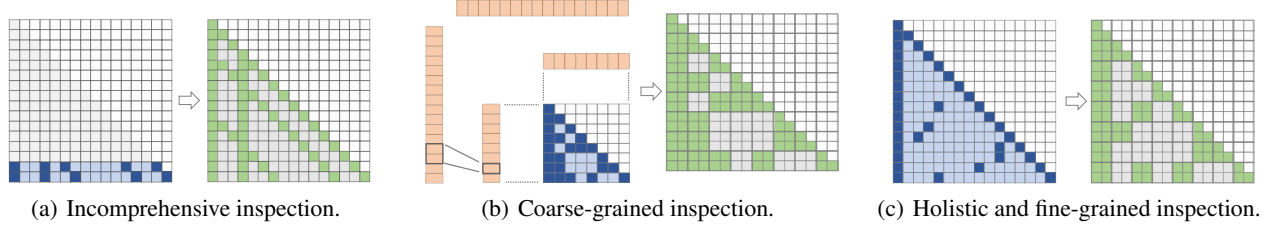


Figure 1. Illustration of attention map inspection mechanisms in sparse attention methods. (a) Overview of MInference: It inspects the attention scores of the query tokens at the very end of the sequence to identify important regions. (b) Overview of SpargeAttn: It selects significant regions based on attention scores derived from representative tokens. (c) Overview of SALE: It performs a comprehensive and fine-grained inspection of the entire attention map.

dict critical attention regions by analyzing the attention map. For instance, as illustrated in Figure 1(a), MInference (Jiang et al., 2024) and SampleAttention (Zhu et al., 2024), decompose the sparse attention pattern into combinations of multiple vertical or slash lines, and predict the positions of these lines by analyzing the attention score distribution of a subset of query tokens. Another series of sparse attention methods, such as FlexPrefill (Lai et al., 2025), SpargeAttn (Zhang et al., 2025d), and HiP Attention (Lee et al., 2025), treat the attention map as the concatenation of blocks. As shown in Figure 1(b), they dynamically skip certain attention computations at the block granularity. These methods often construct a representative token for each consecutive query/key chunk and build sparse masks based on the product between representative tokens. Although existing dynamic sparse attention methods can accelerate the prefilling stage of LLM inference to some extent (Ma et al., 2025), they fail to achieve a satisfactory accuracy-efficiency trade-off, as their inspection approaches are either too coarse-grained or insufficiently comprehensive.

In this paper, we propose **SALE**, a novel block-Sparse Attention technique based on **Low-bit Estimation** of attention weights, to significantly accelerate the long-context prefilling stage of LLM inference with negligible loss in model accuracy. SALE is built on a fast and accurate framework for fine-grained attention map inspection. Figure 1(c) illustrates how SALE inspects the attention map. By performing element-wise importance analysis on the entire attention map, SALE is capable of constructing highly sparse attention masks, while ensuring that the output error is bounded within an acceptable tolerance range. We propose two key components to implement this framework. First, we utilize low-bit quantized query-key products (QK) to approximate attention weights. This process runs efficiently on modern GPUs, leveraging two key factors: the use of high-throughput low-bit Tensor Core instructions and the reduction in global memory access. Second, we propose a novel *Relative Attention Score* metric to evaluate the importance of query-key pairs. Observing that the attention scores in the sink (beginning) and local (end) regions of each attention

map row tend to be relatively higher (Xiao et al., 2024b; Gu et al., 2025), we determine the importance of a query-key pair based on the relative magnitude between its attention weight and the attention weights within the sink and local regions. Compared to common practice that uses the original attention scores (Zhang et al., 2023; Li et al., 2024; Zhang et al., 2025a) as indicators, our design is more efficient because it avoids materializing the entire attention map in DRAM. It also supports adaptive adjustment of the sparsity level according to the input, which allows SALE to robustly handle input samples of varying complexities. SALE further adopts several kernel optimization techniques to optimize hardware efficiency.

SALE incurs no additional training overhead and can be seamlessly integrated into existing inference systems. We conduct comprehensive experiments on various long-context processing benchmarks using two LLMs, Llama-3.1-8B-Instruct (Grattafiori et al., 2024) and Qwen3-4B-Instruct (Yang et al., 2025a), to verify the effectiveness of our method. Experimental results demonstrate that our method delivers a speed-up of at least $3.36\times$ when processing sequences longer than 64K tokens, while maintaining negligible accuracy loss. It achieves superior accuracy-efficiency trade-off compared to the baseline methods.

2. Related works

Sparse LLM inference Many previous works try to leverage the sparsity nature of transformer model to accelerate LLM inference from different perspectives. One line of research exploits input text sparsity to dynamically prune context irrelevant to the user’s query (Jha et al., 2024; Liu et al., 2025; Shi et al., 2024; Jiang et al., 2023; Li et al., 2023b). While these methods can significantly reduce LLM inference latency for relatively simple prompts, they severely degrade generation quality when processing complex inputs (Yuan et al., 2024).

Numerous studies have observed sparsity patterns in self-attention modules, where only a small subset of attention map elements are much larger than the rest. Some meth-

ods (Xiao et al., 2024b; Fu et al., 2025; Xiao et al., 2024a) use predefined static sparsity patterns to prune the attention map. However, these methods suffer from accuracy degradation as the attention sparsity distribution varies among different input contexts (Jiang et al., 2024; Lai et al., 2025). Other methods assume that the distribution follows certain structures, such as Vertical-Slash or Block-Sparse. Some of them (Jiang et al., 2024; Zhu et al., 2024) try to dynamically predict the location of important regions by examining the exact attention scores of several tokens. Others (Gao et al., 2024; Zhang et al., 2025d; Lai et al., 2025; Lee et al., 2025) regard the attention map of compressed tokens, which are generated from continuous token chunks, as the proxy of real attention map. All these methods fail to achieve accurate predictions due to their overly coarse-grained approximations of attention maps.

In contrast to the aforementioned approaches, several alternatives to self-attention have emerged to circumvent its quadratic complexity. Notable examples include: (1) natively sparse attention algorithms (Yuan et al., 2025; Lu et al., 2025), (2) linear attention mechanisms (Peng et al., 2023; Yang et al., 2024b), and (3) state-space models (Gu & Dao, 2024; Dao & Gu, 2024). However, these methods impose significant adoption costs as they require full model training.

During the decoding stage, methods like SparQ (Ribar et al., 2024) and InfiniGen (Lee et al., 2024) compress the channels of query / key tokens to efficiently approximate the attention scores. Retrieval-based approaches (Zhang et al., 2025a; Chen et al., 2025; Liu et al., 2024) leverage vector-retrieval techniques to approximately sort the attention scores of previous input tokens. Several existing algorithms compress tokens by analyzing attention maps during the prefilling stage. These approaches either eliminate redundant tokens (Zhang et al., 2023; Liu et al., 2023; Li et al., 2024; Ge et al., 2024; Devoto et al., 2024) or perform token merging (Zhang et al., 2024b; Zandieh et al., 2024). Our method is orthogonal to these optimizations and can be combined to further enhance end-to-end LLM inference efficiency.

Attention kernel optimization Many CUDA kernel optimization techniques (Dao et al., 2022; Dao, 2024; Shah et al., 2024; Sanovar et al., 2024) leverage hardware features to accelerate the computation of the original full attention. Although these methods accelerate computation, they still require full attention calculations and fail to fully exploit the inherent sparsity of attention maps.

3. Method

3.1. Problem formulation

We denote the query, key and value matrices as Q , K and V , respectively, while the corresponding vectors at token offset i are q_i, k_i, v_i . Let N represent the sequence length and d represent the hidden size. The shapes of Q , K and V are all $N \times d$. Matrix M is the sparse attention mask with a shape of $N \times N$. Single-head self-attention module can be mathematically formalized as below:

$$\text{Attn}(Q, K, V, M) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}} + M\right) \cdot V \quad (1)$$

During the computation of self-attention, the attention weight matrix S is defined as $S = QK^T/\sqrt{d}$, and the attention score matrix P is defined as $P = \text{Softmax}(S + M)$. The sparse attention mask is formed by $M = M_c + M_s$, where $M_c, M_s \in \{0, -\infty\}$ represent the causal mask and the sparse mask respectively. Based on the mathematical properties of Softmax function, if an item $M[i, j]$ in matrix M is $-\infty$, its corresponding attention score will be zero. Therefore, we can skip the attention computation at this position.

For block-sparse attention, query and key tokens are divided into continuous blocks of sizes b_q, b_k along the sequence dimension. We denote the query, key token block at position j as Q_j, K_j , which have the shapes of $b_q \times d$ and $b_k \times d$ respectively. For simplicity, we assume $b_q \mid N, b_k \mid N$, and denote $N_q = N/b_q, N_k = N/b_k$. As shown in Figure 2(a), the attention map can be viewed as the concatenation of $N_q \cdot N_k$ attention blocks, each of shape $b_q \times b_k$. Block sparse attention skips certain computations at the block level. To formulate, we denote $M_{bs} \in \{0, 1\}$ as *block-level sparse mask*, and values of sparse mask M_s depend on M_{bs} :

$$M_s[i, j] = \begin{cases} 0, & \text{if } M_{bs}[\lfloor i/b_q \rfloor, \lfloor j/b_k \rfloor] = 1, \\ -\infty, & \text{if } M_{bs}[\lfloor i/b_q \rfloor, \lfloor j/b_k \rfloor] = 0 \end{cases} \quad (2)$$

In other words, the attention computation between Q_i, K_j, V_j will be skipped if $M_{bs}[i, j]$ is zero. Block-sparse attention aims to maximize sparsity in matrix M_{bs} while bounding the approximation error relative to full attention within a tolerable threshold.

3.2. Block selection via fine-grained importance approximation

Considering the dynamic nature of the attention pattern in LLMs (Jiang et al., 2024; Lai et al., 2025), constructing "Sparse and Accurate" M_{bs} is highly challenging. To achieve this goal, SALE evaluates the "importance" of each position in the attention map. Such a framework enables SALE to examine the attention map at a fine-grained level,

allowing robust handling of various dynamic attention patterns. We propose two key designs—*Low-bit Attention Weight Approximation* and *Relative Importance Approximation*—to ensure the efficient and accurate operation of SALE.

Low-bit attention weight approximation SALE first calculates the attention weights for all query-key pairs. Rather than using full-precision floating-point Q and K matrices, SALE computes attention weights using low-bit quantized versions $\tilde{Q}$ and $\tilde{K}$ for approximation, significantly minimizing additional overhead with high-throughput low-bit Tensor Core instructions and reduced DRAM access. For instance, on RTX 4090 and RTX 5090 GPUs, the throughput of 4-bit and 8-bit Tensor Core instructions is $8\times$ and $4\times$ that of 16-bit (full precision) Tensor Core instructions, respectively (NVIDIA, 2023; 2025). Meanwhile, the overhead of loading 4-bit and 8-bit QK matrices from DRAM is $1/4$ and $1/2$ of that required for loading 16-bit QK matrices, respectively.

Regarding the selection of the quantization scheme, we choose to quantize query-key matrices to 4-bit, which maintains high selection accuracy while maximizing the acceleration capability of the low-bit instructions in GPU Tensor Cores. We provide more detailed discussion about quantization scheme in Section I.

Relative importance approximation Denoting approximated attention weights as $\tilde{S}$, the next step is to evaluate the “importance” of each attention block. In related works (Zhang et al., 2023; Li et al., 2024; Liu et al., 2025; 2023), a commonly used metric is the attention score, obtained by applying *Softmax* function to attention weights. Compared to existing practices, our approach performs importance computation on-the-fly, bypassing the need to compute full attention scores. We propose *Relative Attention Score* as our importance metric instead of the original attention scores.

Our design is based on an observation in many related studies (Xiao et al., 2024b; Gu et al., 2025; Xiao et al., 2024a). As shown in Figure 2(b), attention scores within the “sink and local regions” (i.e. the beginning and end of each row) maintain consistently large values, while the region exhibits consistent size across diverse input sequences. Motivated by this pattern, we assess “importance” by comparing $\tilde{S}[i, j]$ with the attention weights from the sink and local regions. Specifically, before examining those blocks located in the middle of the sequence, we first compute full precision QK matrix multiplication on blocks in the sink and local areas. Denoting the indices of key tokens within the sink and local regions as I_{SL} , this process yields two intermediate values, $\tilde{m}_i$ and $\tilde{l}_i$. We then compute the *Relative Attention Score* $\tilde{P}[i, j]$ base on these two intermediate results, which can be

formulated as follows:

$$\tilde{m}_i = \max_{j \in I_{SL}} S[i, j], \quad \tilde{l}_i = \sum_{j \in I_{SL}} e^{S[i, j] - \tilde{m}_i}$$

$$\tilde{P}[i, j] = \frac{e^{\tilde{S}[i, j] - \tilde{m}_i}}{\tilde{l}_i}$$

If all $\tilde{P}[i, j]$ values in a block are smaller than the threshold τ (e.g. 0.004), this block is marked as non-critical, and the attention computation for this block will be skipped. The procedure for determining the threshold value $\tau \in (0, 1)$ is elaborated in Section 3.3.

3.3. Per-head threshold calibration

Figure 2(b) illustrates the attention score distributions of two attention heads of Llama-3.1-8B-Instruct, exhibiting inconsistent sparsity levels. Thus, applying the same τ for all heads may lead to suboptimal performance. To address the issue, we propose an offline calibration procedure to determine the optimal τ value for each head, which ensures negligible output errors while maximizing sparsity.

We adopt the L_1 distance between the output of SALE and the output of full attention as the error metric, which can be formulated as $Err(\tau) = \|O - \tilde{O}\|_1 / N$. O is the result of the original attention, $\tilde{O}$ is the result of SALE, and N represents sequence length. At the beginning of the calibration, τ is initially set to be a relatively large threshold τ_0 (e.g. 0.008). We then progressively reduce the sparsity level by halving the value of τ until $Err(\tau)$ falls below θ , where θ is the predefined error bound. By tuning θ , we can control the sparsity level of SALE. In Section G, we provide a detailed explanation of the correlations between these hyperparameters.

3.4. Kernel optimization

For kernel implementation, we propose several optimization techniques to further improve the hardware efficiency of SALE. Here, we describe some core designs, and more detailed information about the optimizations can be found in Section H.

Computation tiling SALE targets long context processing scenarios, where materializing the intermediate results of QK GEMM would occupy a large amount of DRAM space and introduce extensive I/O overhead. Therefore, we implement SALE based on the framework of FlashAttention2 (Dao, 2024) to avoid this issue. Specifically, the computation tasks of different query blocks will be scheduled to different cooperative thread arrays (CTA) for parallel processing. Each CTA iteratively checks the importance between its corresponding query block and different key blocks.

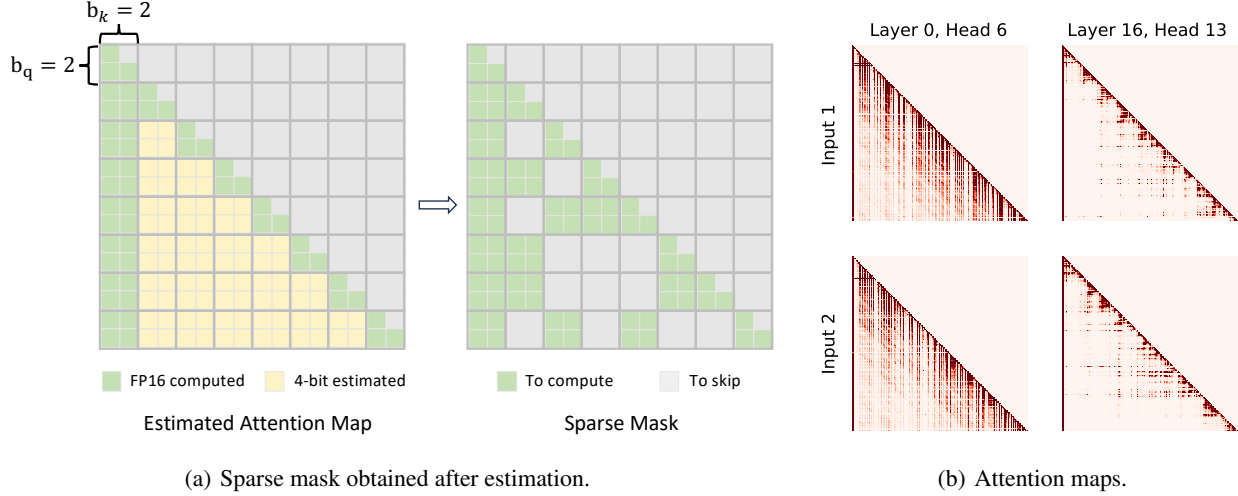


Figure 2. (a) Illustration of SALE. The whole 16×16 attention map is viewed as concatenation of many 2×2 blocks. We first estimate the attention weights in an element-wise manner, and then construct a sparse mask at the block level based on these estimations. (b) Attention maps of two different attention heads in Llama-3.1-8B-Instruct when processing different input sequences.

Estimation-computation disaggregation In consideration of performance issues, SALE separates "important block selection" and "sparse attention computation" into two distinct CUDA kernels, which are referred to as *Selection-Pass* and *Computation-Pass*, respectively. During Selection-Pass, we select important attention regions at the block granularity and record the coordinates of these blocks. We provide the corresponding pseudo code in Section K for a more detailed description. After Selection-Pass, we compute attention output on selected blocks in the following Computation-Pass.

A potentially simpler and more efficient implementation is the "One-pass" alternative: upon identifying an important key block, immediately compute the output for that block. However, such a design is not feasible in practice, as it fails to reduce computational workload on the one hand and is incompatible with the hardware characteristics of GPUs on the other. We provide a more detailed discussion on this issue in Section J.

Relative attention score comparison Directly computing *Relative Attention Score* is time-consuming as it consists of multiple complex hardware instructions, including floating point division and exponential function. Considering that $\tilde{l}_i$ and $\tilde{m}_i$ do not change after computation in sink and local areas, we optimize this comparison by following mathematical transformation:

$$\frac{e^{\tilde{S}[i,j]-\tilde{m}_i}}{\tilde{l}_i} \geq \tau \iff \tilde{S}[i,j] \geq \ln(\tau \cdot \tilde{l}_i + \tilde{m}_i) \quad (3)$$

The comparison between the *Relative Attention Score* and τ can then be accomplished using a single floating point comparison instruction. It is worth noting that we also

mitigate potential overflow issues caused by the exponential function.

Integration with SageAttention The final stage of SALE is Computation-Pass. We employ the QK 8-bit quantization strategy proposed in SageAttention (Zhang et al., 2025c) to further accelerate Computation-Pass while maintaining negligible precision loss.

4. Experiments

4.1. Settings

Models Most of the experiments are conducted using Llama-3.1-8B-Instruct (Grattafiori et al., 2024) (**Llama-3.1**). We also use Qwen3-4B-Instruct (Yang et al., 2025a) (**Qwen-3**) to validate the effectiveness of our method on larger-scale LLM.

Implementation details We implement Selection-Pass in C++ CUDA and use Triton (Tillet et al., 2019) compiler to accelerate the quantization process (Sun et al., 2025). We implement the quantized Computation-Pass based on the open-source code of SpargeAttn (Zhang et al., 2025d). For those hyper-parameters mentioned in Section 3.2, we use block size $b_q = 64$ and $b_k = 32$. We constrain the sink area size to 32 tokens and the local area size to no more than 256 tokens. During offline calibration, we set the initial threshold $\tau_0 = 0.008$, and use error bounds of $\theta = 0.4$ for Llama-3.1 and $\theta = 4.0$ for Qwen-3 by default. All latency experiments are conducted on a server with 8 GeForce RTX 4090 GPUs without using tensor-parallel (Narayanan et al., 2021; Zhang et al., 2025f) or context-parallel (Li et al., 2023a) techniques. In the Section A, we provide additional

Table 1. LongBench evaluation results of different methods. We use **boldface** to denote the highest value and underline to indicate the second-highest value.

Tasks	Llama-3.1					Qwen-3				
	FA2	MInfer	Flex	Sparge	SALE	FA2	MInfer	Flex	Sparge	SALE
NarrativeQA	29.93	24.92	28.29	<u>29.62</u>	28.95	27.99	<u>27.63</u>	23.82	25.29	26.67
Qasper	44.82	44.29	44.55	43.73	45.33	44.40	<u>43.80</u>	37.93	42.31	42.81
MultiFieldQA	54.65	53.71	<u>55.34</u>	56.02	55.18	51.49	50.30	50.30	49.61	<u>51.17</u>
HotpotQA	<u>55.81</u>	52.00	55.38	54.57	55.83	58.30	<u>57.72</u>	54.62	53.17	57.15
2WikiMQA	<u>46.16</u>	44.10	43.43	47.08	42.61	42.51	41.93	35.96	35.84	<u>42.13</u>
MuSiQue	<u>30.41</u>	25.72	30.07	31.40	30.10	<u>22.85</u>	22.17	19.54	23.07	23.12
GovReport	<u>35.29</u>	35.09	34.64	35.04	35.45	30.61	<u>30.56</u>	30.00	29.22	30.42
QMSum	25.25	<u>25.47</u>	25.83	25.12	25.33	<u>22.76</u>	22.66	22.99	22.13	22.73
TREC	72.50	<u>72.00</u>	70.50	71.00	70.50	<u>74.50</u>	75.50	74.00	74.00	74.00
TriviaQA	<u>91.65</u>	91.18	89.81	92.68	90.47	87.99	88.68	87.54	<u>88.51</u>	88.43
SAMSum	43.67	<u>43.73</u>	43.18	43.18	44.19	45.35	45.47	45.25	45.62	<u>45.51</u>
LSHT	46.50	<u>46.00</u>	41.00	45.50	46.50	43.50	41.25	35.50	40.25	<u>42.50</u>
Count	<u>6.72</u>	3.25	2.59	5.89	7.09	<u>1.33</u>	1.08	0.00	0.67	2.58
Retrieval	<u>99.50</u>	97.00	82.00	84.00	100.00	100.00	100.00	56.50	<u>91.50</u>	100.00
Average	48.77	47.03	46.18	47.48	<u>48.39</u>	46.68	46.34	40.99	44.42	<u>46.37</u>
Speedup (64K)	1.00×	1.07×	2.21	<u>3.11×</u>	3.36×	1.00×	1.91×	<u>2.78×</u>	2.55×	3.63×

Table 2. InfiniteBench evaluation results of different methods. We use **boldface** to denote the highest value and underline to indicate the second-highest value.

Tasks	Llama-3.1					Qwen-3				
	FA2	MInfer	Flex	Sparge	SALE	FA2	MInfer	Flex	Sparge	SALE
Retrieve.KV	<u>55.60</u>	20.00	38.00	47.20	56.40	52.00	48.00	58.00	32.00	<u>55.00</u>
En.MC	<u>67.25</u>	55.02	68.56	66.38	66.38	64.00	<u>60.00</u>	59.00	57.00	64.00
Math.Find	34.29	34.86	30.00	<u>34.57</u>	30.57	27.00	27.00	29.00	17.00	<u>28.00</u>
En.QA	15.12	13.96	<u>14.18</u>	13.51	13.19	8.27	7.10	7.71	<u>7.78</u>	7.49
En.Dia	16.50	13.50	17.00	<u>17.50</u>	19.00	8.00	8.00	6.00	<u>7.00</u>	6.00
Average	37.75	27.47	33.55	35.83	<u>37.11</u>	31.85	30.02	<u>31.94</u>	24.14	32.09
Speedup (64K)	1.00×	1.07×	2.21	<u>3.11×</u>	3.36×	1.00×	1.91×	<u>2.78×</u>	2.55×	3.63×

information about the implementation.

Baselines To demonstrate the advantages of SALE, we compare it with four strong baselines for self-attention acceleration in long-context processing: **FlashAttention2**(FA2) (Dao, 2024), **MInference**(MInfer) (Jiang et al., 2024), **FlexPrefill**(Flex) (Lai et al., 2025), and **SpargeAttn**(Sparge) (Zhang et al., 2025d). FA2 computes standard full attention, which serves as an oracle. The other three methods employ sparse attention mechanisms. All experimental results are based on their publicly available implementation. We use $\gamma = 0.95$ for Llama-3.1 and $\gamma = 0.93$ for Qwen-3 when evaluating FlexPrefill. We use $(l_1 = 0.08, l_2 = 0.09)$ for both Llama-3.1 and Qwen-3 when evaluating SpargeAttn. For MInference, we select the optimal sparse pattern based on its open-source code. Additionally, to investigate the performance of these methods under varying sparsity levels, we prepare multiple sets of hyperparameters based on their publicly available codes. For FlexPrefill and SpargeAttn, as described in their papers, we adjust their sparsity levels by tuning γ and (l_1, l_2) , respectively. For MInference, since its open-source implementation configures all heads with the *Vertical-Slash* pattern,

the sparsity rate is adjusted by varying the total number of vertical and slash lines across all heads. We provide more details in Section B.

Metrics To validate the effectiveness of SALE, we assess model quality using long-context benchmarks (see Section 4.2) and measure efficiency through latency measurements. All latency results in the experimental section focus solely on the attention computation time across all layers during the LLM prefilling phase. Our latency measurements include all online operations, such as quantization, block selection, and index selection. In some experiments, we report the end-to-end (E2E) latency on certain datasets, which is computed by summing the latency of all samples in the dataset.

4.2. Accuracy evaluation

Following common practice (Zhang et al., 2025d; Jiang et al., 2024; Lai et al., 2025; Zhang et al., 2025a; Gao et al., 2024; Li et al., 2024), we adopt three long-context understanding benchmarks to compare the generation quality of our method with other baselines. Achieving higher scores

Table 3. RULER evaluation results of different methods. We use **boldface** to denote the highest value and underline to indicate the second-highest value.

Context length	Llama-3.1					Qwen-3				
	FA2	MInfer	Flex	Sparge	SALE	FA2	MInfer	Flex	Sparge	SALE
8K	92.10	<u>92.52</u>	91.44	92.68	91.68	95.46	<u>95.95</u>	92.77	71.06	96.04
16K	89.56	87.41	90.54	<u>89.83</u>	89.51	<u>94.27</u>	95.09	93.49	81.99	93.62
32K	84.43	84.17	85.74	84.15	<u>85.22</u>	93.17	91.42	91.63	81.33	<u>92.68</u>
64K	80.81	<u>81.54</u>	80.46	80.03	82.12	90.24	87.26	88.68	86.22	<u>90.10</u>
128K	72.43	61.41	<u>72.57</u>	71.91	74.81	83.82	82.17	80.46	80.45	<u>83.47</u>
Average	83.86	81.41	<u>84.15</u>	83.72	84.67	91.39	90.38	89.41	80.21	<u>91.18</u>
Speedup (64K)	1.00×	1.07×	2.21	<u>3.11×</u>	3.36×	1.00×	1.91×	<u>2.78×</u>	2.55×	3.63×

on these baselines indicates better performance. (1) **LongBench** (Bai et al., 2024): A comprehensive benchmark covering diverse long-text applications, including single-document QA, multi-document QA, summarization, few-shot learning, synthetic tasks, etc. The context lengths of most input samples are below 32K tokens. (2) **InfiniteBench** (Zhang et al., 2024a): A benchmark designed to evaluate the capability of processing excessively long context (exceeding 100K tokens). It comprises several challenging synthetic tasks such as Retrieve.KV and Math.Find, as well as other real-world tasks including QA and summarization based on fake books or fake dialogues. (3) **RULER** (Hsieh et al., 2024): A widely used synthetic long-context benchmark, including tasks such as Needle-In-A-Haystack and Multi-hop Tracing. Both task difficulty and context length can be flexibly configured.

LongBench Table 1 presents the LongBench evaluation results comparing SALE with baseline approaches. In the second row of the table, we use abbreviations introduced in Section 4.1 to denote each method. In the last two rows, we report the average scores and the latency speedup. The latency metric, measured on a 64K-length Needle-In-A-Haystack input, confirms the premise of our accuracy test, that SALE achieves the highest speedup when processing long contexts.

The results on both models show that SALE achieves superior accuracy among all sparse attention baselines.

InfiniteBench Table 2 presents the test scores of InfiniteBench, evaluating the capability of processing extremely long inputs. As shown in the table, our method also achieves the best accuracy-efficiency trade-off on InfiniteBench. In addition, when our method is applied, Llama-3.1 exhibits only marginal performance degradation while Qwen-3 even shows improvement. We attribute this improvement to our method’s ability to potentially filter noisy information during the prefilling phase, thereby enhancing the model’s comprehension capabilities.

RULER Table 3 presents the evaluation results on the RULER benchmark. We compare our method against various sparse attention baselines across different context lengths and report the average score for each approach. As shown in the table, our method consistently outperforms other sparse attention baselines in terms of both efficiency and accuracy.

4.3. Efficiency evaluation

Single input speedup We first compare the latency of different methods when processing a single input. The results are presented in Figure 3(a). We conduct experiments using Llama-3.1 and report the speedup of each method relative to FlashAttention2. To illustrate how latency scales with the number of tokens, we prepare four input samples of different lengths. These samples are obtained by truncating a single 128K-length input from the Needle-In-A-Haystack task in RULER.

Our method demonstrates consistent speedups over FlashAttention2 across all sequence lengths while outperforming all sparse attention baselines in most cases. At 16K context length, SALE achieves a 1.99× speedup. It further exhibits greater speedup as context length increases—reaching 3.88× at 128K—owing to sparser attention patterns. It should be noted that the hyperparameters of each sparse method in this test are aligned with those in Section 4.2, indicating that our speedup evaluation is based on SALE achieving optimal accuracy performance.

Accuracy vs efficiency We adjust the computation budget of each method following the approach described in Section 4.1 to analyze the accuracy-efficiency trade-offs. Considering that the speedup achieved by dynamic sparse attention methods may vary depending on the input content, we evaluate the end-to-end latency of all methods on LongBench, InfiniteBench and RULER for comprehensive comparison. The results, shown in Figure 4, demonstrate the superior performance of our method on these datasets.

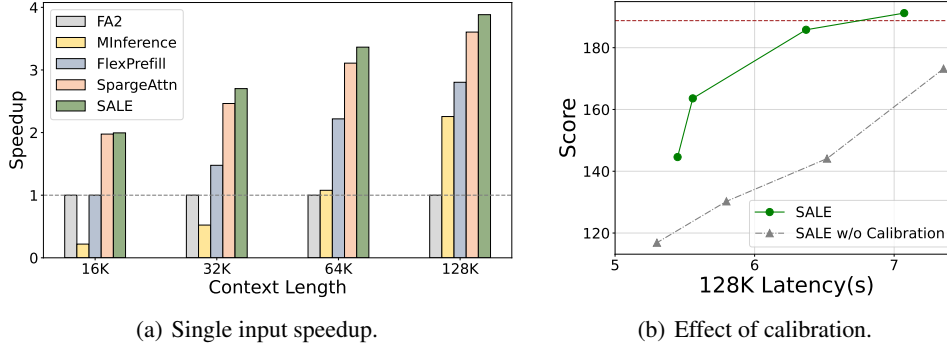


Figure 3. (a) Speedup in single-input processing. (b) Comparison between SALE v.s. SALE w/o Calibration on InfiniteBench. The brown horizontal dashed line represents the score achieved by FlashAttention2.

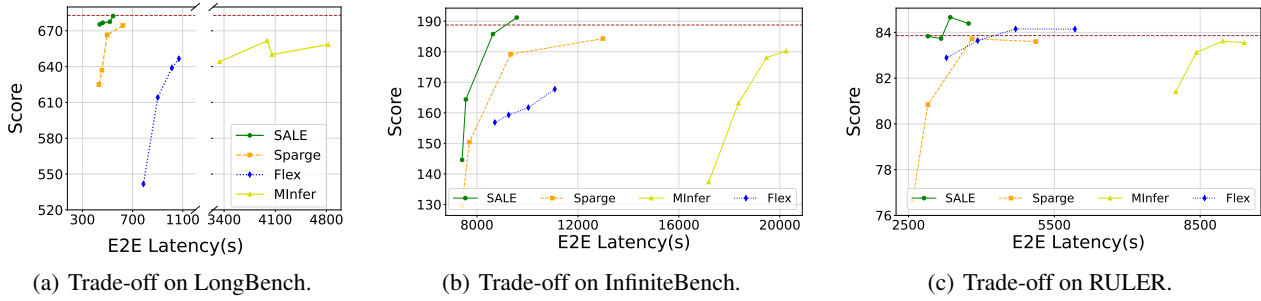


Figure 4. Evaluation of accuracy-efficiency trade-offs. The brown horizontal dashed line represents the score achieved by FlashAttention2. Tradeoff curves demonstrates that SALE achieves the best speed-accuracy trade-off across all three datasets.

4.4. Ablation study

In this section, we evaluate the latency of each stage in SALE and assess the impact of per-head threshold calibration. Additionally, in Section D and Section I, we provide additional experimental results to demonstrate the block selection accuracy of SALE. We also present sparsity measurement results of these sparse methods across contexts of varying lengths in Section F, which serves as a supplement to our previous comparison.

Latency breakdown We report the latency breakdown results of SALE under various input lengths in Table 4. All experiments use Llama-3.1, with reported timings reflecting end-to-end execution across all 32 model layers. In the second-to-last line, we show the execution time ratio of Quantization and Selection-Pass operations relative to full attention latency. In the final line, we present the speedup of Computation-Pass compared to full attention. The results demonstrate that our method introduces acceptable computational overhead, with its relative cost decreasing as sequence length grows. Furthermore, Computation-Pass shows greater speedups with longer context lengths, reflecting improved sparsity level at scale.

Threshold calibration To demonstrate the performance gain brought by per-head threshold calibration, we set all

Table 4. Latency breakdown (ms).

Context length	8K	16K	32K	64K	128K
Quantization	11	21	47	99	208
Selection-Pass	14	48	166	634	2562
Computation-Pass	51	137	378	1117	3599
FA2	106	416	1597	6224	24731
Overhead ratio	23.9%	16.7%	13.3%	11.5%	11.1%
Computation-Pass speedup	2.08×	3.04×	4.23×	5.57×	6.87×

heads in Llama-3.1 to share the same τ , which is referred to as *SALE w/o Calibration*. As shown in Figure 3(b), setting different τ values according to the characteristics of attention patterns across different heads yields substantial performance gains.

5. Conclusion

In this paper, we propose a block-Sparse Attention technique based on Low-bit Estimation. By performing fine-grained estimation of the attention map, we achieve a better accuracy-efficiency trade-off. Specifically, we estimate the attention weights using low-bit quantized queries and keys, and assess the importance of query-key pairs using our *Relative Attention Score* metric. Furthermore, we introduce several CUDA kernel optimization techniques to ensure the

efficiency of sparse mask construction on hardware. Experimental results demonstrate that our approach achieves the best trade-off among existing sparse attention baselines, delivering a speedup of at least $3.36\times$ when processing sequences longer than 64K tokens while maintaining negligible accuracy loss.

Limitation

This work targets acceleration of the prefill phase for long-context LLM inference, but it shows limited effectiveness on two workload categories. When input sequences are shorter than roughly 8K tokens, attention computation itself incurs little cost. The additional overhead from low-bit estimation exceeds the savings achieved by block skipping, resulting in no tangible end-to-end advantage compared with full attention. For decoding-bound workloads characterized by short prompts and lengthy generation, the prefill stage accounts for minimal overall runtime, and its optimization cannot mitigate user-perceived latency. We note that decoding-stage acceleration is an orthogonal research direction and is not covered in this paper.

Due to our method’s reliance on high-throughput 4-bit Tensor Core instructions to accelerate the Selection-Pass, it may lose its performance advantage on hardware that does not support efficient 4-bit matrix multiplication. Moreover, our current implementation is limited to approximating attention weights using Int4 quantization. Additional adaptations would be needed to deploy our method on hardware that supports FP4 GEMM or LUT-based low-bit GEMM. We leave it as our future work.

Acknowledgements

This work is supported by National Natural Science Foundation of China (U23B2048, 62402011), Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (JYB2025XDXM108), and High-performance Computing Platform of Peking University.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench: A bilingual, multitask benchmark for long

context understanding. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL <https://aclanthology.org/2024.acl-long.172/>.

Beltagy, I., Peters, M. E., and Cohan, A. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Caciularu, A., Peters, M., Goldberger, J., Dagan, I., and Cohan, A. Peek across: Improving multi-document modeling via cross-document question-answering. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1970–1989, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.110. URL <https://aclanthology.org/2023.acl-long.110/>.

Chang, Y., Lo, K., Goyal, T., and Iyyer, M. Boookscore: A systematic exploration of book-length summarization in the era of LLMs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=7Ttk3RzDeu>.

Chen, Z., Sadhukhan, R., Ye, Z., Zhou, Y., Zhang, J., Nolte, N., Tian, Y., Douze, M., Bottou, L., Jia, Z., and Chen, B. MagicPIG: LSH sampling for efficient LLM generation. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=ALzTQUgW8a>.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations*, volume 2024, pp. 35549–35562, 2024.

Dao, T. and Gu, A. Transformers are ssms: generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Dai, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Bao, H., Xu, H., Wang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Wang, J., Chen, J., Chen, J., Yuan, J., Qiu, J., Li, J., Song, J., Dong, K., Hu, K., Gao, K., Guan, K., Huang, K., Yu, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Wang, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Wang, Q., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Zhang, R., Pan, R., Wang, R., Xu, R., Zhang, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Pan, S., Wang, T., Yun, T., Pei, T., Sun, T., Xiao, W. L., Zeng, W., Zhao, W., An, W., Liu, W., Liang, W., Gao, W., Yu, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Zhang, X., Chen, X., Nie, X., Sun, X., Wang, X., Cheng, X., Liu, X., Xie, X., Liu, X., Yu, X., Song, X., Shan, X., Zhou, X., Yang, X., Li, X., Su, X., Lin, X., Li, Y. K., Wang, Y. Q., Wei, Y. X., Zhu, Y. X., Zhang, Y., Xu, Y., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Yu, Y., Zheng, Y., Zhang, Y., Shi, Y., Xiong, Y., He, Y., Tang, Y., Piao, Y., Wang, Y., Tan, Y., Ma, Y., Liu, Y., Guo, Y., Wu, Y., Ou, Y., Zhu, Y., Wang, Y., Gong, Y., Zou, Y., He, Y., Zha, Y., Xiong, Y., Ma, Y., Yan, Y., Luo, Y., You, Y., Liu, Y., Zhou, Y., Wu, Z. F., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Xu, Z., Huang, Z., Zhang, Z., Xie, Z., Zhang, Z., Hao, Z., Gou, Z., Ma, Z., Yan, Z., Shao, Z., Xu, Z., Wu, Z., Zhang, Z., Li, Z., Gu, Z., Zhu, Z., Liu, Z., Li, Z., Xie, Z., Song, Z., Gao, Z., and Pan, Z. Deepseek-v3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.
- Deng, Y., Song, Z., Xiong, J., and Yang, C. Attention is naturally sparse with gaussian distributed input. *CoRR*, abs/2404.02690, 2024. doi: 10.48550/ARXIV.2404.02690. URL <https://doi.org/10.48550/arXiv.2404.02690>.
- Devoto, A., Zhao, Y., Scardapane, S., and Minervini, P. A simple and effective l_2 norm-based strategy for KV cache compression. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 18476–18499, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.1027. URL <https://aclanthology.org/2024.emnlp-main.1027/>.
- Fan, A., Jernite, Y., Perez, E., Grangier, D., Weston, J., and Auli, M. Eli5: Long form question answering, 2019. URL <https://arxiv.org/abs/1907.09190>.
- Fu, T., Huang, H., Ning, X., Zhang, G., Chen, B., Wu, T., Wang, H., Huang, Z., Li, S., Yan, S., Dai, G., Yang, H., and Wang, Y. Mixture of attention spans: Optimizing LLM inference efficiency with heterogeneous sliding-window lengths. In *Second Conference on Language Modeling*, 2025. URL <https://openreview.net/forum?id=n3rZJrWPLE>.
- Fu, Y. Challenges in deploying long-context transformers: A theoretical peak performance analysis, 2024. URL <https://arxiv.org/abs/2405.08944>.
- Gao, Y., Zeng, Z., Du, D., Cao, S., Zhou, P., Qi, J., Lai, J., So, H. K.-H., Cao, T., Yang, F., et al. Seerattention: Learning intrinsic sparse attention in your llms. *arXiv preprint arXiv:2410.13276*, 2024.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=uNrFpDPMYo>.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., van der Linde, J., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnston, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhota, K., Rantala-Young, L., van der Maaten, L., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., de Oliveira, L., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi, N.,

- Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damla, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A. L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu, S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. The llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=tEYskw1VY2>.
- Gu, X., Pang, T., Du, C., Liu, Q., Zhang, F., Du, C., Wang, Y., and Lin, M. When attention sink emerges in language models: An empirical view. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=78Nn4QJTEN>.
- Han, C., Wang, Q., Peng, H., Xiong, W., Chen, Y., Ji, H., and Wang, S. Lm-infinite: Zero-shot extreme length generalization for large language models. In *NAACL-HLT*, pp. 3991–4008, 2024. URL <https://doi.org/10.18653/v1/2024.naacl-long.222>.
- Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekish, D.,

- Jia, F., Zhang, Y., and Ginsburg, B. Ruler: What's the real context size of your long-context language models?, 2024. URL <https://arxiv.org/abs/2404.06654>.
- Jha, S., Erdogan, L. E., Kim, S., Keutzer, K., and Ghohami, A. Characterizing prompt compression methods for long context inference. *arXiv preprint arXiv:2407.08892*, 2024.
- Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., and Qiu, L. LLM-Lingua: Compressing prompts for accelerated inference of large language models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://openreview.net/forum?id=ADsEdyI32n>.
- Jiang, H., Li, Y., Zhang, C., Wu, Q., Luo, X., Ahn, S., Han, Z., Abdi, A. H., Li, D., Lin, C.-Y., Yang, Y., and Qiu, L. Minference 1.0: accelerating pre-filling for long-context llms via dynamic sparse attention. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.
- Kryściński, W., Rajani, N., Agarwal, D., Xiong, C., and Radev, D. Booksum: A collection of datasets for long-form narrative summarization, 2022. URL <https://arxiv.org/abs/2105.08209>.
- Lai, X., Lu, J., Luo, Y., Ma, Y., and Zhou, X. Flexpre-fill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=OfjIlbelrT>.
- Lee, H., Park, G., Lee, Y., Suh, J., Kim, J., Jeong, W., Kim, B., Lee, H., Jeon, M., and Hwang, S. J. A training-free sub-quadratic cost transformer model serving framework with hierarchically pruned attention. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=PTcMzQgKmn>.
- Lee, W., Lee, J., Seo, J., and Sim, J. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 155–172, 2024.
- Li, D., Shao, R., Xie, A., Xing, E. P., Ma, X., Stoica, I., Gonzalez, J. E., and Zhang, H. Distflashattn: Distributed memory-efficient attention for long-context llms training. *arXiv preprint arXiv:2310.03294*, 2023a.
- Li, Y., Dong, B., Lin, C., and Guerin, F. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*, 2023b.
- Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H., Cai, T., Lewis, P., and Chen, D. Snapkv: Llm knows what you are looking for before generation. *Advances in Neural Information Processing Systems*, 37:22947–22970, 2024.
- Lin, Y., Chen, H., Xia, W., Lin, F., Wang, Z., and Liu, Y. A comprehensive survey on deep learning techniques in educational data mining: A comprehensive survey on deep learning techniques. lin et al. *Data Science and Engineering*, 10:564–590, 07 2025. doi: 10.1007/s41019-025-00303-z.
- Liu, D., Chen, M., Lu, B., Jiang, H., Han, Z., Zhang, Q., Chen, Q., Zhang, C., Ding, B., Zhang, K., et al. Retrievalattention: Accelerating long-context llm inference via vector retrieval. *arXiv preprint arXiv:2409.10516*, 2024.
- Liu, J., Chen, B., and Zhang, C. Speculative prefill: Turbocharging TTFT with lightweight and training-free token importance estimation. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=bzbuZ0ItBq>.
- Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., and Shrivastava, A. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. *Advances in Neural Information Processing Systems*, 36:52342–52364, 2023.
- Lu, E., Jiang, Z., Liu, J., Du, Y., Jiang, T., Hong, C., Liu, S., He, W., Yuan, E., Wang, Y., et al. MoBA: Mixture of Block Attention for Long-Context LLMs. *arXiv preprint arXiv:2502.13189*, 2025.
- Ma, L. X., Wang, L. P., Shao, E., Cao, R. Y., and Tan, G. M. vastpipe: A high-throughput inference system via adaptive space-division multiplexing for diverse accelerators. *Journal of Computer Science and Technology*, 40(2):444–463, 2025. ISSN 1000-9000(Print) /1860-4749(Online). doi: 10.1007/s11390-024-3773-5. URL <https://jcst.ict.ac.cn/en/article/doi/10.1007/s11390-024-3773-5>.
- Narayanan, D., Shoeybi, M., Casper, J., LeGresley, P., Patwary, M., Korthikanti, V., Vainbrand, D., Kashinkunti, P., Bernauer, J., Catanzaro, B., Phanishayee, A., and Zaharia, M. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '21, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450384421. doi: 10.1145/3458817.3476209. URL <https://doi.org/10.1145/3458817.3476209>.

- NVIDIA. Nvidia ada gpu architecture. Technical report, NVIDIA, 2023. URL <https://images.nvidia.cn/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>.
- NVIDIA. Nvidia rtx blackwell gpu architecture. Technical report, NVIDIA, 2025. URL <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>.
- Pang, R. Y., Parrish, A., Joshi, N., Nangia, N., Phang, J., Chen, A., Padmakumar, V., Ma, J., Thompson, J., He, H., and Bowman, S. R. Quality: Question answering with long input texts, yes!, 2022. URL <https://arxiv.org/abs/2112.08608>.
- Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Derczynski, L., Du, X., Grella, M., GV, K. K., He, X., Hou, H., Kazienko, P., Kocon, J., Kong, J., Koptyra, B., Lau, H., Lin, J., Mantri, K. S. I., Mom, F., Saito, A., Song, G., Tang, X., Wind, J. S., Wozniak, S., Zhang, Z., Zhou, Q., Zhu, J., and Zhu, R.-J. RWKV: Reinventing rnns for the transformer era. In *EMNLP (Findings)*, pp. 14048–14077, 2023. URL <https://doi.org/10.18653/v1/2023.findings-emnlp.936>.
- Porwal, S., Bewoor, L., and Deshpande, V. Transformer based implementation for automatic book summarization. *arXiv preprint arXiv:2301.07057*, 2023.
- Ribar, L., Chelombiev, I., Hudlass-Galley, L., Blake, C., Luschi, C., and Orr, D. Sparq attention: Bandwidth-efficient LLM inference. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=OS5dqxmmtl>.
- Sanovar, R., Bharadwaj, S., Amant, R. S., Rühle, V., and Rajmohan, S. Lean attention: Hardware-aware scalable attention mechanism for the decode-phase of transformers. *arXiv preprint arXiv:2405.10480*, 2024.
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37:68658–68685, 2024.
- Shi, Z., Ming, Y., Nguyen, X.-P., Liang, Y., and Joty, S. Discovering the gems in early layers: Accelerating long-context llms with 1000x input token reduction. *arXiv preprint arXiv:2409.17422*, 2024.
- Sun, S., Bai, J., Chen, H., Deng, K., Xie, Z., Li, J., Cao, B., Zhang, H., Kang, W., and Zhao, W. Model quantization for computing-in-memory: a survey. *Science China Information Sciences*, 68, 2025. URL <https://api.semanticscholar.org/CorpusID:282833001>.
- Team, G., Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., Perrin, S., Matejovicova, T., Ramé, A., Rivière, M., Rouillard, L., Mesnard, T., Cideron, G., bastien Grill, J., Ramos, S., Yvinec, E., Casbon, M., Pot, E., Penchev, I., Liu, G., Visin, F., Kenealy, K., Beyer, L., Zhai, X., Tsitsulin, A., Busa-Fekete, R., Feng, A., Sachdeva, N., Coleman, B., Gao, Y., Mustafa, B., Barr, I., Parisotto, E., Tian, D., Eyal, M., Cherry, C., Peter, J.-T., Sinopalnikov, D., Bhupatiraju, S., Agarwal, R., Kazemi, M., Malkin, D., Kumar, R., Vilar, D., Brusilovsky, I., Luo, J., Steiner, A., Friesen, A., Sharma, A., Sharma, A., Gilady, A. M., Goedeckemeyer, A., Saade, A., Feng, A., Kolesnikov, A., Bendebury, A., Abdagic, A., Vadi, A., György, A., Pinto, A. S., Das, A., Bapna, A., Miech, A., Yang, A., Paterson, A., Shenoy, A., Chakrabarti, A., Piot, B., Wu, B., Shahriari, B., Petrini, B., Chen, C., Lan, C. L., Choquette-Choo, C. A., Carey, C., Brick, C., Deutsch, D., Eisenbud, D., Cattle, D., Cheng, D., Paparas, D., Sreepathihalli, D. S., Reid, D., Tran, D., Zelle, D., Noland, E., Huizenga, E., Kharitonov, E., Liu, F., Amirkhanyan, G., Cameron, G., Hashemi, H., Klimczak-Plucińska, H., Singh, H., Mehta, H., Lehri, H. T., Hazimeh, H., Ballantyne, I., Szpektor, I., Nardini, I., Pouget-Abadie, J., Chan, J., Stanton, J., Wieting, J., Lai, J., Orbay, J., Fernandez, J., Newlan, J., yeong Ji, J., Singh, J., Black, K., Yu, K., Hui, K., Vodrahalli, K., Greff, K., Qiu, L., Valentine, M., Coelho, M., Ritter, M., Hoffman, M., Watson, M., Chaturvedi, M., Moynihan, M., Ma, M., Babar, N., Noy, N., Byrd, N., Roy, N., Momchev, N., Chauhan, N., Sachdeva, N., Bunyan, O., Botarda, P., Caron, P., Rubenstein, P. K., Culliton, P., Schmid, P., Sessa, P. G., Xu, P., Stanczyk, P., Tafti, P., Shivanna, R., Wu, R., Pan, R., Rokni, R., Willoughby, R., Vallu, R., Mullins, R., Jerome, S., Smoot, S., Girgin, S., Iqbal, S., Reddy, S., Sheth, S., Pöder, S., Bhatnagar, S., Panyam, S. R., Eiger, S., Zhang, S., Liu, T., Yacovone, T., Liechty, T., Kalra, U., Evci, U., Misra, V., Roseberry, V., Feinberg, V., Kolesnikov, V., Han, W., Kwon, W., Chen, X., Chow, Y., Zhu, Y., Wei, Z., Egyed, Z., Cotruta, V., Giang, M., Kirk, P., Rao, A., Black, K., Babar, N., Lo, J., Moreira, E., Martins, L. G., Sanseviero, O., Gonzalez, L., Gleicher, Z., Warkentin, T., Mirrokni, V., Senter, E., Collins, E., Barral, J., Ghahramani, Z., Hadsell, R., Matias, Y., Sculley, D., Petrov, S., Fiedel, N., Shazeer, N., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Alayrac, J.-B., Anil, R., Dmitry, Lepikhin, Borgeaud, S., Bachem, O., Joulin, A., Andreev, A., Hardin, C., Dadashi, R., and Hussenot, L. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.

- Tillet, P., Kung, H. T., and Cox, D. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, pp. 10–19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367196. doi: 10.1145/3315508.3329973. URL <https://doi.org/10.1145/3315508.3329973>.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wang, C., Zhang, J., Feng, Y., Li, T., Sun, W., Liu, Y., and Peng, X. Teaching code llms to use autocompletion tools in repository-level code generation, 2024a. URL <https://arxiv.org/abs/2401.06391>.
- Wang, Y., Wang, Y., Guo, D., Chen, J., Zhang, R., Ma, Y., and Zheng, Z. RlCoder: Reinforcement learning for repository-level code completion, 2024b. URL <https://arxiv.org/abs/2407.19487>.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
- Xiao, G., Tang, J., Zuo, J., Guo, J., Yang, S., Tang, H., Fu, Y., and Han, S. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024a.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=NG7sS51zVF>.
- Xu, W., Huang, C., Gao, S., and Shang, S. Llm-based agents for tool learning: A survey. *Data Science and Engineering*, 10:533 – 563, 2025. URL <https://api.semanticscholar.org/CorpusID:279637071>.
- Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., et al. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115*, 2024a.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025a. URL <https://arxiv.org/abs/2505.09388>.
- Yang, A., Yu, B., Li, C., Liu, D., Huang, F., Huang, H., Jiang, J., Tu, J., Zhang, J., Zhou, J., et al. Qwen2. 5-1m technical report. *arXiv preprint arXiv:2501.15383*, 2025b.
- Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training. In *Forty-first International Conference on Machine Learning*, 2024b. URL <https://openreview.net/forum?id=ia5XvxFUJT>.
- Yuan, J., Liu, H., Zhong, S., Chuang, Y.-N., Li, S., Wang, G., Le, D., Jin, H., Chaudhary, V., Xu, Z., et al. Kv cache compression, but what must we give in return? a comprehensive benchmark of long context capable approaches. *arXiv preprint arXiv:2407.01527*, 2024.
- Yuan, J., Gao, H., Dai, D., Luo, J., Zhao, L., Zhang, Z., Xie, Z., Wei, Y., Wang, L., Xiao, Z., et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.
- Zaheer, M., Guruganesh, G., Dubey, K. A., Ainslie, J., Alberti, C., Ontanon, S., Pham, P., Ravula, A., Wang, Q., Yang, L., et al. Big bird: Transformers for longer sequences. *Advances in neural information processing systems*, 33:17283–17297, 2020.
- Zandieh, A., Han, I., Mirrokni, V., and Karbasi, A. Subgen: Token generation in sublinear time and memory. *arXiv preprint arXiv:2402.06082*, 2024.
- Zhang, H., Ji, X., Chen, Y., Fu, F., Miao, X., Nie, X., Chen, W., and Cui, B. PQCache: Product Quantization-based KVCache for long context LLM inference. *Proc. ACM Manag. Data*, 3(3), June 2025a. doi: 10.1145/3725338. URL <https://doi.org/10.1145/3725338>.
- Zhang, J., Huang, H., Zhang, P., Wei, J., Zhu, J., and Chen, J. SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference*

- on Machine Learning, volume 267 of *Proceedings of Machine Learning Research*, pp. 75097–75119. PMLR, 13–19 Jul 2025b. URL <https://proceedings.mlr.press/v267/zhang25ae.html>.
- Zhang, J., wei, J., Zhang, P., Zhu, J., and Chen, J. Sageattention: Accurate 8-bit attention for plug-and-play inference acceleration. In *The Thirteenth International Conference on Learning Representations*, 2025c. URL <https://openreview.net/forum?id=OL44KtasKc>.
- Zhang, J., Xiang, C., Huang, H., Wei, J., Xi, H., Zhu, J., and Chen, J. SpargeAttention: Accurate and training-free sparse attention accelerating any model inference. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 76397–76413. PMLR, 13–19 Jul 2025d. URL <https://proceedings.mlr.press/v267/zhang25ch.html>.
- Zhang, X., Chen, Y., Hu, S., Xu, Z., Chen, J., Hao, M. K., Han, X., Thai, Z. L., Wang, S., Liu, Z., and Sun, M. ∞ bench: Extending long context evaluation beyond 100k tokens, 2024a. URL <https://arxiv.org/abs/2402.13718>.
- Zhang, Y., Du, Y., Luo, G., Zhong, Y., Zhang, Z., Liu, S., and Ji, R. Cam: Cache merging for memory-efficient llms inference. In *Forty-first International Conference on Machine Learning*, 2024b.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- Zhang, Z., Wang, J., Wen, F., Guo, Y., Zhao, X., Fang, X., Ding, S., Jia, Z., Xiao, J., Shen, Y., Zheng, Y., Zhu, X., Wu, Y., Jiao, Z., Sun, W., Chen, Z., Zhang, K., Fu, K., Cao, Y. S., Hu, M., Zhou, Y., Zhou, X., Cao, J., Zhou, W., Cao, J., Li, R., Zhou, D., Tian, Y., Zhu, X., yuan Li, C., Wu, H., Liu, X., He, J., Zhou, Y., Liu, H., Zhang, L., Wang, Z., Duan, H., Zhou, Y., Min, X., Jia, Q., Zhou, D., Zhang, W., Cao, J., Yang, X., Yu, J., Zhang, S., Duan, H., and Zhai, G. Large multimodal models evaluation: a survey. *Science China Information Sciences*, 68, 2025e. URL <https://api.semanticscholar.org/CorpusID:283102393>.
- Zhang, Z. X., Wen, Y. B., Lyu, H. Q., Liu, C., Zhang, R., Li, X. Q., Wang, C., Du, Z. D., Guo, Q., Li, L., Zhou, X. H., and Chen, Y. J. Ai computing systems for large language models training. *Journal of Computer Science and Technology*, 40(1):6–41, 2025f. ISSN 1000-9000(Print) /1860-4749(Online). doi: 10.1007/s11390-024-4178-1. URL <https://jcst.ict.ac.cn/en/article/doi/10.1007/s11390-024-4178-1>.
- Zhu, Q., Duan, J., Chen, C., Liu, S., Li, X., Feng, G., Lv, X., Cao, H., Chuanfu, X., Zhang, X., et al. Sampleattention: Near-lossless acceleration of long context llm inference with adaptive structured sparse attention. *arXiv preprint arXiv:2406.15486*, 2024.

A. Additional implementation details

For the quantization algorithm scheme, we use the smoothing technique mentioned in SageAttention2 (Zhang et al., 2025b) to improve the quality of QK quantization, which introduce negligible overhead. While the quality of quantization affects the accuracy performance of SALE, it should be noted that SALE is designed to be orthogonal to the quantization algorithm.

We select five input samples from the Retrieve.KV task in InfiniteBench to perform calibration for SALE, and the final configuration must satisfy the error bound requirement across all five samples. The per-head threshold calibration for Llama-3.1 on RTX4090 server takes approximately five minutes to complete.

B. Additional experiment details

For model inference, we leverage the transformers (Wolf et al., 2020) library to build an execution pipeline and replace the default self-attention module with sparse methods. We use greedy decoding to avoid randomness during generation, and use the default chat template to construct the input prompt. We use the same set of input samples as our method to search the optimal hyperparameters for SpargeAttention. Since the open-source code of MInference only uses one input sample for calibration, we employed the first sample from this set for its sparse pattern searching.

During evaluation process, to ensure proper model behavior, we truncate samples that exceed the maximum context window length. Following common practice, we retain the tokens from both the beginning and the end of the sequence and remove those from the middle portion. For all these benchmarks and tasks, we employ the official evaluation scripts from their respective open-source repositories to assess model outputs.

For the data format during model inference, we employed BFloat16 for FlexPrefill due to requirements specified in its repository, while Float16 was used for all other methods.

C. Performance comparison on larger model

To provide a more comprehensive comparison with other methods, we evaluated the performance of each approach based on the Qwen-2.5-32B-Instruct model (Yang et al., 2024a). Table 5 and Table 6 demonstrate the evaluation results. Evaluation results show that SALE’s advantages persist even with larger models.

Table 5. LongBench evaluation results of different methods. We use **boldface** to denote the highest value and underline to indicate the second-highest value.

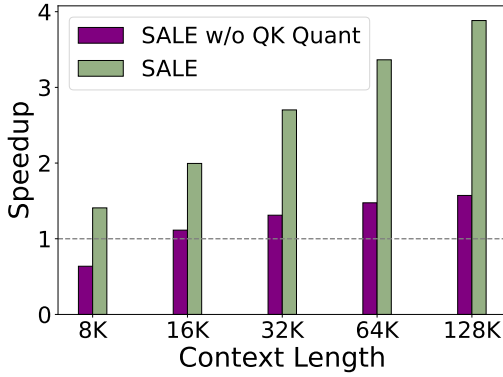
Tasks	FA2	MInfer	Qwen-2.5		SALE
			Flex	Sparge	
NarrativeQA	29.20	<u>31.27</u>	29.80	29.19	32.21
Qasper	<u>45.79</u>	45.05	45.53	44.61	45.95
MultiFieldQA	<u>53.25</u>	53.01	52.61	51.66	53.37
HotpotQA	<u>64.68</u>	64.59	64.78	63.94	63.95
2WikiMQA	60.87	60.82	62.98	61.13	<u>62.33</u>
MuSiQue	39.89	41.38	39.46	39.22	<u>40.54</u>
GovReport	30.38	30.59	30.78	30.36	<u>30.66</u>
QMSum	23.06	23.16	23.10	<u>23.18</u>	23.42
TREC	<u>73.50</u>	<u>73.50</u>	<u>73.50</u>	74.50	73.00
TriviaQA	87.68	88.40	89.40	<u>88.81</u>	87.97
SAMSum	45.67	45.92	46.43	<u>46.41</u>	45.92
LSHT	45.79	47.50	44.17	47.00	<u>47.21</u>
Count	12.67	13.67	3.57	9.22	<u>13.38</u>
Retrieval	99.50	<u>99.25</u>	92.25	98.83	98.25
Average	50.85	<u>51.29</u>	49.88	50.57	51.30
Speedup (64K)	1.00×	1.25×	1.39×	<u>2.55×</u>	3.28×

D. 4bit Selection-Pass vs 16bit Selection Pass

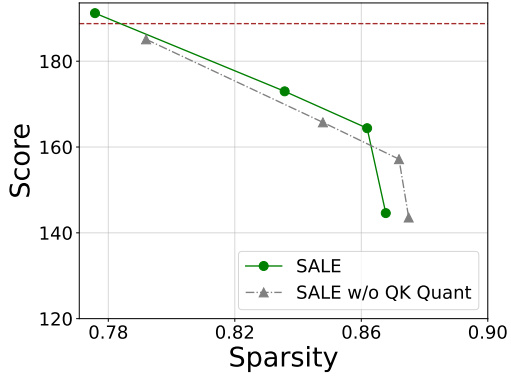
To evaluate the effectiveness of 4-bit attention weight approximation, we further conducted experiments using original-precision (16-bit) QK matrices to inspect the attention map, which is referred to as *SALE w/o QK Quant*. The result is shown in Figure 5. We measure the single input speedup of two methods under varying input lengths, using the same set of input

Table 6. InfiniteBench evaluation results of different methods. We use **boldface** to denote the highest value and underline to indicate the second-highest value.

Tasks	FA2	MInfer	Qwen-2.5		
			Flex	Sparge	SALE
Retrieve.KV	4.00	7.60	4.80	4.60	<u>5.40</u>
En.MC	63.70	<u>63.32</u>	59.80	61.50	62.80
Math.Find	41.40	45.71	<u>47.20</u>	41.70	52.00
En.QA	6.70	<u>6.85</u>	6.70	6.80	6.90
En.Dia	27.50	30.00	<u>29.00</u>	25.50	27.50
Average	28.66	<u>30.70</u>	29.5	28.02	30.92
Speedup (64K)	1.00×	1.25×	1.39×	<u>2.55×</u>	3.28×



(a) Single input speedup



(b) Accuracy and sparsity result

Figure 5. Comparison between SALE and SALE w/o QK Quant. (a) Single input speedup. (b) Comparison between SALE v.s. SALE w/o QK Quant on InfiniteBench. The brown horizontal dashed line represents the score achieved by FlashAttention2.

samples as in the Figure 3(a). The result indicates that using original-precision QK to estimate attention weights leads to a significant increase in computational overhead.

We further evaluate the accuracy and attention sparsity of both methods based on Llama-3.1, where corresponding data points for the two methods are obtained using the same θ . We use the scores from InfiniteBench to represent accuracy. Attention sparsity metric is defined as the ratio of the number of skipped attention blocks to the total number of attention blocks, and the results presented here are measured when processing contexts of 128K length. As observed, under identical hyperparameter settings, *SALE w/o QK Quant* achieves higher attention sparsity while showing a slight performance drop on InfiniteBench. This may be attributed to the limited precision of current Int4 quantization techniques, which can cause certain approximated attention weights to exceed their true values, thereby leading to more blocks being selected.

E. Latency comparison with quantized full attention

To clarify how much of SALE’s speedup comes from sparsity rather than from low-bit attention itself, we compare SALE against SageAttention (Zhang et al., 2025c), an INT8-quantized full-attention scheme. We measure single-input attention latency on Llama-3.1-8B-Instruct using a single RTX 4090, sweeping context length from 8K to 128K. SALE uses hyperparameter $\theta = 0.4$ throughout.

Table 7. Attention latency (ms) comparison

Method	8K	16K	32K	64K	128K
Full (FP16)	107	423	1642	6404	25657
Sage (INT8)	78	285	1076	3994	15592
SALE ($\theta = 0.4$)	73	203	592	1876	6524
Speedup over Sage	1.06×	1.40×	1.82×	2.13×	2.39×

Although SageAttention already provides a substantial speedup over FlashAttention2, SALE delivers an additional $1.06\times-2.39\times$ speedup on top, with the gap widening as context grows. This confirms that block skipping based on low-bit estimation is complementary to attention quantization. Even with quantized dense kernels, sparsity exploitation yields considerable further gains in long-context settings.

F. Sparsity statistic analysis

To enable a more transparent comparison between SALE and other sparse attention baselines introduced in the Section 4, we tested the sparsity results of these methods on the input samples from LongBench. We divided all inputs into three groups according to their context lengths and computed the average sparsity rate along with its standard deviation for each group during processing. The hyperparameters used for each sparse method here are consistent with those described in Section 4.1. The results are in Table 8, with data in each cell formatted as (mean, std):

Table 8. Sparsity comparison

Sparse methods	MInfer	Flex	Sparge	SALE
0-20K	(0.138, 0.029)	(0.604, 0.035)	(0.184, 0.016)	(0.517, 0.028)
20K-40K	(0.356, 0.043)	(0.676, 0.046)	(0.279, 0.022)	(0.651, 0.028)
> 40K	(0.471, 0.044)	(0.677, 0.026)	(0.339, 0.025)	(0.678, 0.020)

The experimental results show that our scheme has obvious advantages in terms of sparsity rate compared with MInference and SpargeAttn. When compared with FlexPrefill, our sparsity rate is lower for context length $\leq 40K$, but higher for context length $> 40K$.

It should be noted that a higher sparsity rate of FlexPrefill here does not mean better performance. According to the results in Figure 4(a), SALE can achieve both higher scores and faster end-to-end latency than FlexPrefill under various sparsity rates.

G. Hyperparameters explanation

τ controls the sparsity rate of a specific attention head. When the relative attention score of an attention block is less than τ , this block will be ignored in the Computation Pass. Therefore, as τ increases, more blocks will be skipped, leading to a higher sparsity rate, decreased accuracy, and faster attention computation. The value of τ can vary across different heads, and its specific value is determined during the calibration process.

τ_0 represents the initial value of τ in calibration process for each attention head. It will gradually decrease during the calibration process until the output error requirement is met. Therefore, it is sufficient for τ_0 to have a relatively large initial value. τ_0 does not affect the speedup and accuracy performance of the model.

θ , on the other hand, is a hyperparameter that adjusts the global sparsity rate. When θ increases, the τ values for each head will also increase, and the sparsity level of SALE get higher. In our Figure 4, we have assessed the performance of SALE under various values of θ .

H. Additional Kernel optimization technique

Reduction in dequantization operations Theoretically, whether an attention block is skipped only depends on the comparison between the largest *Relative Attention Score* with τ . By employing per-thread quantization strategy proposed in (Zhang et al., 2025b), we make all quantized attention weight elements held by each thread share the same quantization scale. This ensures that the largest *Relative Attention Score* and the largest approximated attention weight occur at the same position. Therefore, only the largest approximated attention weight needs to be dequantized, which saves many low-throughput operations such as datatype conversion.

Segment level all-reduce Due to the hardware characteristics of GPU Tensor Cores, the QK GEMM for an attention block is collectively executed by multiple threads within a GPU thread blocks with the output of the QK GEMM stored in a distributed manner across different threads. Since the elements held by each thread are invisible to other threads, we need to perform a CTA-wise all-reduce operation on the relative importance comparison results in each thread. This ensures that all

threads within the thread blocks reach a unified judgment on the importance of the current block.

In fact, this all-reduce operation is costly: it introduces multiple `warp_shfl` instructions, `_syncthreads` instructions, and shared memory access instructions. Since the Selection-Pass does not require computing attention output, we employ specific optimization techniques to reduce the number of all-reduce operations. Specifically, each thread can temporarily store the relative importance of multiple consecutive blocks as multiple bits in local variables, and use a single all-reduce operation to achieve CTA-wise consistency. This allows us to save a significant number of all-reduce operations.

I. Quantization scheme analysis

SALE uses quantized QK to estimate attention weights in Selection-Pass. Theoretically, the fewer the number of bits used in quantization, the faster the estimation of attention weights will be, as this allows the use of faster Tensor Core instructions and results in lower DRAM memory access overhead. Considering that the minimum bit width supported by Tensor Cores in current mainstream computing cards is 4-bit, the most efficient quantization scheme we can choose is 4-bit quantization for QK. Lower quantization bit widths cannot yield additional performance benefits.

However, lower-precision quantization also introduces greater errors, possibly leading to a decrease in the accuracy of important block selection in Selection-Pass. To demonstrate the block selection accuracy under 4-bit quantization, We tested the “recall rate” between the blocks selected using 4-bit QK and those selected using full precision floating-point QK. The “recall rate” metric can be defined as “the number of blocks commonly selected by both methods” divided by “the total number of blocks selected using full precision QK”. The input samples used in the test are aligned with those in Figure 3(a), and the tests are also based on the Llama-3.1-8B-Instruct model. We obtained the statistical results under different context lengths, which are demonstrated in Table 9:

Table 9. Block selection recall rate

Context length	8K	16K	32K	64K	128K
Recall rate	98.2%	97.7%	97.2%	96.6%	95.4%

The experimental results demonstrate that 4-bit QK is capable of accurately selecting important blocks. The “Sparsity-Accuracy” curve presented in Figure 5(b) also indicates that quantizing QK to 4-bit can achieve block selection accuracy comparable to that of full-precision QK. Thus, SALE adopts 4-bit scheme which enables it to balance accuracy and efficiency.

J. Two pass design rationale

In contrast to our “Estimation-computation disaggregation” (“Two-pass”) kernel design mentioned in Section 3.4, the “One-pass” design integrates these two processes into a single kernel. Specifically, when iterating through a key block, if the current attention block is determined to be important, the output for the current block is computed directly (i.e. “on-the-fly pruning”).

Theoretically, this design eliminates the need to store the coordinates of important blocks, and can reuse the computation results of low-bit QK GEMM, which is expected to run faster. However, in practice, the “One-pass” design is not feasible. Two primary reasons are listed here:

4-bit quantization issue In fact, the results of 4-bit QK GEMM cannot be directly used for computing attention output, as this would lead to significant accuracy degradation. To illustrate this point, in Table 10, we present the accuracy experiment results of using 4-bit QK in the computation stage. We refer to this scheme as “4+4”, whereas SALE corresponds to “4+8”. We also tested a scheme that calculates attention scores using only 4-bit QK without block pruning, denoted as “4 dense”. All 4-bit quantization in the table has adopted the technique mentioned in SageAttention2 (Zhang et al., 2025b) to improve quantization quality. We report the scores on two tasks of InfiniteBench in Table 10:

Our experimental results show that the “4+4” scheme leads to a degradation in model performance. Even if we do not prune any blocks, a significant score drop still occurs. Therefore, we choose to use QK with a precision of at least 8 bits in the Computation Pass.

Table 10. Accuracy performance comparison

Tasks	Retrieve.KV	En.MC
4+4	35.8	63.30
4 dense	41.4	60.26
SALE (4+8)	56.4	66.38

Implementation challenges According to the hardware characteristics of GPUs, “on-the-fly pruning” cannot be efficiently implemented on GPUs. The requirement for immediate decisions imposed by the “on-the-fly pruning” means that we have to perform an all-reduce operation each time after computing the QK GEMM. Based on the analysis provided in Section H, it will introduce much more CTA-wise all-reduce operations compared to SALE, which is not efficient.

In addition, “on-the-fly pruning” is unfriendly to the GPU memory hierarchy. Due to the high DRAM access latency of GPUs, high-performance CUDA implementations typically pre-issue memory access instructions for data that will be computed to achieve overlap between memory access and computation. However, the nature of “on-the-fly pruning” dictates that we must decide whether to issue the memory access instruction only after completing the QK GEMM, and then wait for the data transmission of value tensor, which waste the computational resource on SM. Otherwise, if we issue memory access instructions for the value tensor for all blocks, the unnecessary memory access will also impair performance.

Based on the above analysis, even if we adopt the “8+8” scheme to attempt reusing the results of 8-bit QK GEMM, it still cannot match the efficiency of “Two-pass” design.

K. Selection-Pass algorithm

Algorithm 1 Selection-Pass

Input: $Q, K \in \mathbb{R}^{N \times d}$, 4-bit quantized matrices $\tilde{Q}, \tilde{K} \in \mathbb{Z}^{N \times d}$, threshold τ , block size b_q, b_k , local area size l .

```

1  $N_q \leftarrow N/b_q, N_k \leftarrow N/b_k, N_{local} \leftarrow l/b_k$  Split  $Q, K$  into blocks  $Q_i \in \mathbb{R}^{b_q \times d}, K_j \in \mathbb{R}^{b_k \times d}$ , split  $\tilde{Q}, \tilde{K}$  into blocks  $\tilde{Q}_i \in \mathbb{Z}^{b_q \times d}, \tilde{K}_j \in \mathbb{Z}^{b_k \times d}$  for  $i = 0$  to  $N_q - 1$  do
2    $I_{SL} \leftarrow \{0\} \cup [i - N_{local}, i - 1]$ ; // Block indices of sink-local area
3    $\tilde{m}, \tilde{l} \in \mathbb{R}^{b_q}, \tilde{m} \leftarrow -\infty, \tilde{l} \leftarrow 0$ ; // Initialize intermediate result
4   for  $j \in I_{SL}$  do
5     if  $j \neq 0$  then
6        $\tilde{m}_{\Delta} \leftarrow \tilde{m} - \text{rowmax}(Q_i K_j^T / \sqrt{d}); \tilde{l} \leftarrow \tilde{l} \cdot \exp(\tilde{m}_{\Delta})$ 
7     end
8      $\tilde{m} \leftarrow \text{rowmax}(Q_i K_j^T / \sqrt{d});$  // Ignore causal mask
9      $\tilde{l} \leftarrow \tilde{l} + \text{rowsum}(\exp(\frac{Q_i K_j^T}{\sqrt{d}} - \tilde{m}))$   $M_{bs}[i, j] \leftarrow 1$ 
10  end
11  for  $j \leftarrow 1$  to  $(i - N_{local} - 1)$  do
12     $\tilde{S}_{ij} \leftarrow \text{Dequantize}(\tilde{Q}_i \tilde{K}_j^T) / \sqrt{d};$  // Approximate attention weight
13     $\tilde{P}_{ij} \leftarrow \exp(\tilde{S}_{ij} - \tilde{m}) / \tilde{l};$  // Compute Relative Attention Score
14     $M_{bs}[i, j] \leftarrow \max(\tilde{P}_{ij}) \geq \tau$ 
15  end
16 end
Output: Block-level sparse mask  $M_{bs}$ 

```